

MVA Track — how to actually use this

A two-year plan from freshman engineering to M2 MVA readiness. 105 weeks, 4 tracks, 860 checkable items. This file explains how to get value out of it, and what to do about hosting.

1. The mental model

The site is **not** a to-do list you clear. It is a dependency graph with a calendar stapled to it.

Two rules explain almost every design decision:

1. **Order matters more than volume.** Nothing appears before the thing that makes it comprehensible. If you jump ahead, the later material becomes memorisation instead of understanding — which is exactly the failure mode that makes MVA courses feel impossible.
2. **Amber means load-bearing.** Items with an amber left border and a `gate` tag are prerequisites. Everything downstream silently assumes them. Skipping a normal item costs you that item. Skipping a gate costs you the next six months.

860 items over 105 weeks is roughly 8 per week. If that feels light, remember the robotics track assumes you're already spending 12+ h/week in the club — the plan schedules that time, it doesn't add to it.

2. The eight pages, and what each is for

Dashboard (/)

Your weekly home. Open this, not the roadmap.

- **Overall ring** — completion across every finishable item.
- **Per-track bars** — completion, planned h/week, and percentage per track. Watch the *gaps between* the bars, not the absolute numbers. If Math is at 40% and CS is at 12%, you are doing the comfortable track and avoiding the important one. That's the single most likely way this plan fails.
- **Current week card** — the week's tasks grouped by track, tickable inline.
- **Hours panel** — type what you actually worked, per track. Commits when you click away or press Enter. Rounds to the nearest half hour.

Before 14 Sep 2026 it shows "Week 1 begins in N days" instead of faking a current week.

Roadmap (/roadmap)

The full 105 weeks in 8 phases. Tap a phase to expand its weeks. Use this for **orientation and recovery**, not daily work — when you've lost the thread, come here to see where you are.

- **Break weeks** (French Zone C holidays) are deliberately lighter, project-focused weeks, not empty ones.
- **Summer weeks** (Aug 2027, Jul–Sep 2028) are marked optional. If you do nothing in August the plan still works; it just gets tighter afterwards.
- `TICK PHASE` / `CLEAR` bulk-toggle a whole phase. Useful when starting mid-plan or writing off a bad month honestly.

Planner (/planner)

The bridge between the plan and the week you actually have. Your timetable isn't fixed yet, and a schedule you can't follow is worse than no schedule — so **this writes nothing to any calendar**.

It works the other way round: you tell it when you're free, it tells you what to do then.

- **The grid** — 7 days x half-hours, 06:00 to 23:00. Click or drag to paint the hours you're genuinely free. It starts pre-filled with the rhythm you described (weeknights 20:00–23:00, weekend mornings and afternoons), which is a guess — correct it. Availability is stored **per week**, so a heavy week and a dead week can look completely different, and `copy w{n-1}` clones last week when nothing changed.
- **What comes back** — your next block of undone work, packed into those hours: gates first, one track per block instead of four context switches, and each track held roughly to its weekly budget. Tasks longer than 2h are split into numbered sessions so they can actually fit a weeknight.
- **Tick tasks straight from here**. Doing so removes them and re-flows the rest of the week.
- **DID NOT FIT** is the honest part. If the same gate keeps landing there, your free hours are the problem, not your ordering — either free more time or accept the plan slips.

Changes save automatically. The schedule itself is never stored; it's recomputed from your availability and whatever is still undone, so it can't go stale.

It's also a CS exercise, deliberately. Packing many tasks into many blocks is *multiple knapsack*, which is NP-hard. One block on its own is a plain 0/1 knapsack, exactly solvable by DP in $O(n \cdot C)$. So the code solves each block exactly, greedily across blocks — optimal per block, a good week rather than a provably optimal one. The recurrence, the value function, and the honest caveats are written up at the bottom of the page, and `bun run scripts/scheduler.test.ts` checks the DP against brute-force subset enumeration on 300 random instances. Read `packages/web/src/web/lib/scheduler.ts` — it's commented as teaching code.

Track pages (</track/math> , </cs> , </ml> , </rob>)

The concept checklists — what each track requires you to *know*, grouped by module and ordered by dependency.

Use the strictest possible tick standard: tick a concept only when you could explain it to someone else, without notes, on a whiteboard. That is the standard MVA exams and technical interviews both apply. Ticking on "I read it and it made sense" makes the whole progress number worthless, and you'll only find out in an exam.

Each track page also states where you start and where you finish, which is worth re-reading when you feel like you're going nowhere.

Resources (</resources>)

80 resources, rated 1–5 **for you specifically** — prépa maths, weak code, zero ML, aiming at MVA in Sep 2028. Not rated for fame. This is why some famous things score 3 and some obscure things score 5.

Filter by track, type, or rating. Search titles, authors, and verdicts. [MARK STARTED](#) tracks what you've picked up — deliberately excluded from your completion percentage, because starting a book is not progress.

Read the verdicts. Where two resources overlap, the verdict says which to actually use and why the other is still listed. Example: CS231n is rated 4 because its videos are patchy — use EECS 498 for lectures and CS231n for the assignments. That kind of thing saves you weeks.

Robotics (</robotics>)

Seven builds, sequenced so each lands right after the maths that makes it non-trivial, plus a €10,000 budget tracked to the line (currently €9,757 planned, €243 slack).

Expand a project for its milestones and full BOM. The slack is intentional: something will cost more than listed, and a budget with no slack forces you to cancel a project rather than absorb a surprise.

Every build is also a portfolio artefact **and** a data source for the ML track — the same robots produce the datasets you train on. That coupling is the point.

Interviews (</interview>)

One milestone per phase, running continuously from Phase 2 to the end.

You asked for fluency, not competence — different skills, different training. Competence is solving the problem; fluency is solving it out loud, under time, while someone watches. Only repetition spread over years produces the second one. Two sessions a week for 100 weeks beats any four-week

grind before applications.

MVA (/mva)

20 courses from the real 2026 catalogue, mapped backwards: for each course, which blocks of this plan make it sittable, and the week after which you meet its prerequisites. Eight are starred — the shortlist matching a vision-plus-robot-learning profile. MVA requires 8 validated modules.

Course names and teachers drift year to year. The value here is the prerequisite mapping, not the catalogue.

3. A working rhythm that fits the plan

Weekly (~20 min, Sunday): open the Dashboard. Log the week's hours per track. Tick what you finished. Read next week's card and note which items are gates. If a gate is coming, protect time for it.

Monthly: open /roadmap and compare your per-track percentages against each other. Fix the laggard before adding anything new.

Quarterly (phase boundary): each phase ends with something showable — a repo, a benchmark, a robot doing a thing. Ship it before moving on. Check the interview milestone for that phase. Then open /mva and confirm the courses you want are still on track.

When you fall behind: you will. Don't clear the backlog item by item. Open the phase, decide what genuinely no longer matters, CLEAR it, and restart from the current week. Preserve gates ruthlessly; drop non-gates without guilt. A plan you've silently abandoned is worse than one you've consciously trimmed.

4. Progress, and where it lives

Progress is stored in **your browser's** localStorage — no account, no server, nothing that can go down. Ticks, logged hours and availability grids all live under the mva: key prefix in whichever browser you're using.

Three consequences worth knowing:

- **Nothing syncs on its own.** Your desktop and your laptop keep separate tick histories. This is the trade you accepted for free, permanent hosting.
- **Use the backup card** at the bottom of the dashboard to move state between machines: export json on the machine you've been ticking from, import json on the other. Import replaces local

state wholesale, so export first if both machines have unsaved progress.

- **Clearing site data wipes your progress.** "Clear cookies and site data" for the domain, or a hard browser reset, takes the tick history with it. Export occasionally — at phase boundaries is a natural rhythm. Keep the JSON with your notes.

The same card has a `reset` button if you ever want to start from zero deliberately.

Four ID schemes, all stable, so an old export always imports cleanly:

`w{week}-{track}-{n}` for week tasks · `c-{track}-{group}-{n}` for concepts · `car-m1` -style for robotics milestones · `iv-p1` ... `iv-p8` for interview milestones.

5. Hosting

You picked **GitHub Pages with progress in `localStorage`**, and that conversion is done. What changed:

- **The database is no longer the source of truth.** All state reads and writes go through `src/web/lib/store.ts`. The hooks in `src/web/stores/` are drop-in replacements for the old server-backed ones, so the pages themselves didn't change.
- **The oRPC API, Drizzle schema and Turso tables are still there**, untouched and still working under `bun run dev`. They're marked in-file as no longer authoritative, and they're what you'd build on if you ever do want cross-device sync. Nothing was deleted.
- **Assets build to relative paths** (`base: "/"`) and routing is **hash-based** (`/#/planner`). Together that means the site works at any repo name, any subpath, even opened from `file://`, with zero configuration — and deep links never 404 because they never reach a server. The URLs have a `#` in them; for a personal tool that's the whole cost.
- **Manual sync** via export/import JSON on the dashboard, since you said you'd synchronise the two machines yourself.

Publishing it

The repo is initialised and committed locally. Three commands and you're live:

```
cd mva-roadmap
git remote add origin git@github.com:<you>/mva-roadmap.git
git branch -M main
git push -u origin main
```

Then in the repo on GitHub: **Settings** → **Pages** → **Build and deployment** → **Source: "GitHub Actions"**.

That's the only setting to touch. `.github/workflows/deploy.yml` is already committed; it installs,

typechecks, builds `packages/web`, and publishes `dist` on every push to `main`. No secrets and no environment variables are needed, because there's no backend to configure.

Your site lands at `https://<you>.github.io/mva-roadmap/`. Free and permanent, and it outlives Runnable — which was the point.

About the preview URL

The port-4200 URL you've been clicking is a dev server inside a Runnable sandbox. It's a **preview**, not hosting: it lives as long as the sandbox does. I don't control sandbox retention, so I can't give you a number of days. Once Pages is up, that's your real URL.

6. Running it locally

```
bun install
bun run dev          # http://localhost:4200
```

No `db:push` and no `.env` needed anymore — the app reads and writes the browser, not a database. (If you ever revive the API for cross-device sync, `bun run db:push` still creates its tables.)

Other commands: `bun run lint`, `bun run typecheck`, `bun run build`, `bun run kill:port`.

Editing the plan

The curriculum is static TypeScript in `packages/web/src/web/content/` — no CMS, no database content. Edit these and the site follows:

File	What's in it
<code>weeks-p1/p2/p3.ts</code>	the 105 weeks. <code>!</code> prefix = gate, <code>@id</code> suffix = resource refs
<code>concepts.ts</code>	248 concepts per track
<code>resources.ts</code>	80 resources with ratings and verdicts
<code>robotics.ts</code>	7 projects, milestones, BOM lines
<code>mva.ts</code>	course map
<code>interview.ts</code>	phase milestones
<code>tracks.ts</code>	track blurbs and weekly hour budgets

One hard rule: task IDs are generated from position ($w_{\{n\}}-{\text{track}}-\{i\}$) and are used as storage keys. Adding weeks or appending tasks is safe. **Reordering or deleting existing tasks silently reassigns other items' progress.** If you must reorder, clear that week's progress first.